

5a) Installation of KVM and creation of Virtual instances

Aim:

To install a KVM and creation of virtual instance for setting up a virtualized environment on linux system.

What is KVM?

KVM (Kernel-based Virtual Machine) is an open-source virtualization technology built into the Linux kernel. It turns Linux into a type-1 hypervisor that allows you to run multiple virtual machines (VMs) on a physical host.

Each VM has its own virtual hardware (CPU, RAM, disk, etc.), and runs its own full-fledged operating system.

Benefits of KVM

Feature	Benefit
Open Source	Free and supported by the Linux community.
Performance	Near-native performance using hardware virtualization (Intel VT or AMD-V).
Security	Isolation between VMs using SELinux and AppArmor.
Scalability	Can handle many VMs on a single host (scales well vertically).
Flexibility	Supports many OSes (Linux, Windows, BSD, etc.).
Integration	Works with tools like libvirt, virt-manager, OpenStack, etc.
Live Migration	Supports live VM migration between hosts.

Procedure:

Virtualization Technology

1. Check Hardware Virtualization Support.

```
keshoo@tce-VirtualBox:~$ egrep -c 'lm' /proc/cpuinfo
4
keshoo@tce-VirtualBox:~$
```

Output ≥ 1 indicates support. If 0, CPU doesn't support hardware virtualization.

2. Run `uname -a`, which shows kernel name, hostname, kernel version, build date, architecture, and OS type in one line.

```
keshoo@tce-VirtualBox:~$ uname -a
Linux tce-VirtualBox 5.4.0-150-generic #167~18.04.1-Ubuntu SMP Wed May 24 00:51:42 UTC 2023 x86_64 x86_64 x86_64 GNU/Linux
keshoo@tce-VirtualBox:~$
```

3. To list the KVM kernel module files present in your kernel, run:

```
tce@tce-VirtualBox:~$ ls /lib/modules/5.4.0-84-generic/kernel/arch/x86/kvm/kvm*
/lib/modules/5.4.0-84-generic/kernel/arch/x86/kvm/kvm-amd.ko
/lib/modules/5.4.0-84-generic/kernel/arch/x86/kvm/kvm-intel.ko
/lib/modules/5.4.0-84-generic/kernel/arch/x86/kvm/kvm.ko
tce@tce-VirtualBox:~$
```

KVM Installation

1. Install required packages : `qemu-kvm`, `libvirt-bin`, `bridge-utils`, `virt-manager` and `qemu-system`.

```
keshoo@tce-VirtualBox:~$ sudo apt-get install qemu-kvm libvirt-bin bridge-utils
qemu-system virt-manager
[sudo] password for keshoo:
Reading package lists... Done
Building dependency tree
Reading state information... Done
bridge-utils is already the newest version (1.5-15ubuntu1).
libvirt-bin is already the newest version (4.0.0-1ubuntu8.21).
qemu-kvm is already the newest version (1:2.11+dfsg-1ubuntu7.42).
qemu-system is already the newest version (1:2.11+dfsg-1ubuntu7.42).
virt-manager is already the newest version (1:1.5.1-0ubuntu1.2).
0 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
keshoo@tce-VirtualBox:~$
```

2. Configure the libvirt daemon configuration file for remote access & socket permissions in KVM/libvirt

```
tce@tce-VirtualBox:~$ sudo nano /etc/libvirt/libvirtd.conf
tce@tce-VirtualBox:~$
```

```
keshoo@tce-VirtualBox:~$ sudo nano /etc/libvirt/libvirtd.conf
[sudo] password for keshoo:
keshoo@tce-VirtualBox:~$ grep ^[#] /etc/libvirt/libvirtd.conf
listen_addr = "0.0.0.0"
unix_sock_group = "libvirt"
unix_sock_ro_perms = "0777"
unix_sock_rw_perms = "0777"
unix_sock_dir = "/var/run/libvirt"
auth_unix_ro = "none"
auth_unix_rw = "none"
keshoo@tce-VirtualBox:~$
```

After configuring it, restart libvirtd service.

```
tce@tce-VirtualBox:~$ sudo systemctl restart libvirtd
tce@tce-VirtualBox:~$
```

3. Run virsh list - Displays no of Virtual machine running.

```
keshoo@tce-VirtualBox:~$ virsh list
 Id      Name                                     State
-----
keshoo@tce-VirtualBox:~$
```

4. Connecting to Virtualization interactive terminal.

```
keshoo@tce-VirtualBox:~$ virsh
Welcome to virsh, the virtualization interactive terminal.

Type:  'help' for help with commands
       'quit' to quit

virsh #
```

5. Getting Version of libvirt & QEMU.

```
virsh # version
Compiled against library: libvirt 4.0.0
Using library: libvirt 4.0.0
Using API: QEMU 4.0.0
Running hypervisor: QEMU 2.11.1

virsh #
```

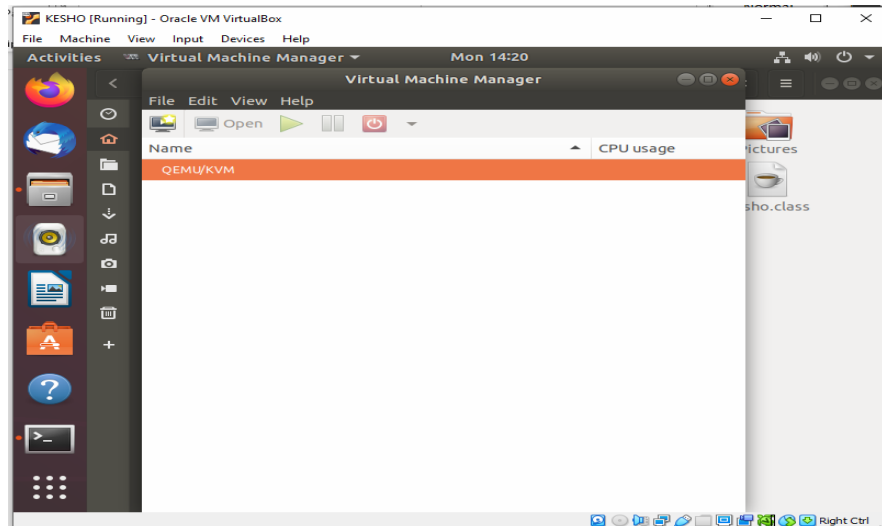
6. Getting Node information

```
virsh # nodeinfo
CPU model:      x86_64
CPU(s):         4
CPU frequency:  3000 MHz
CPU socket(s):  1
Core(s) per socket: 4
Thread(s) per core: 1
NUMA cell(s):  1
Memory size:    4686168 KiB

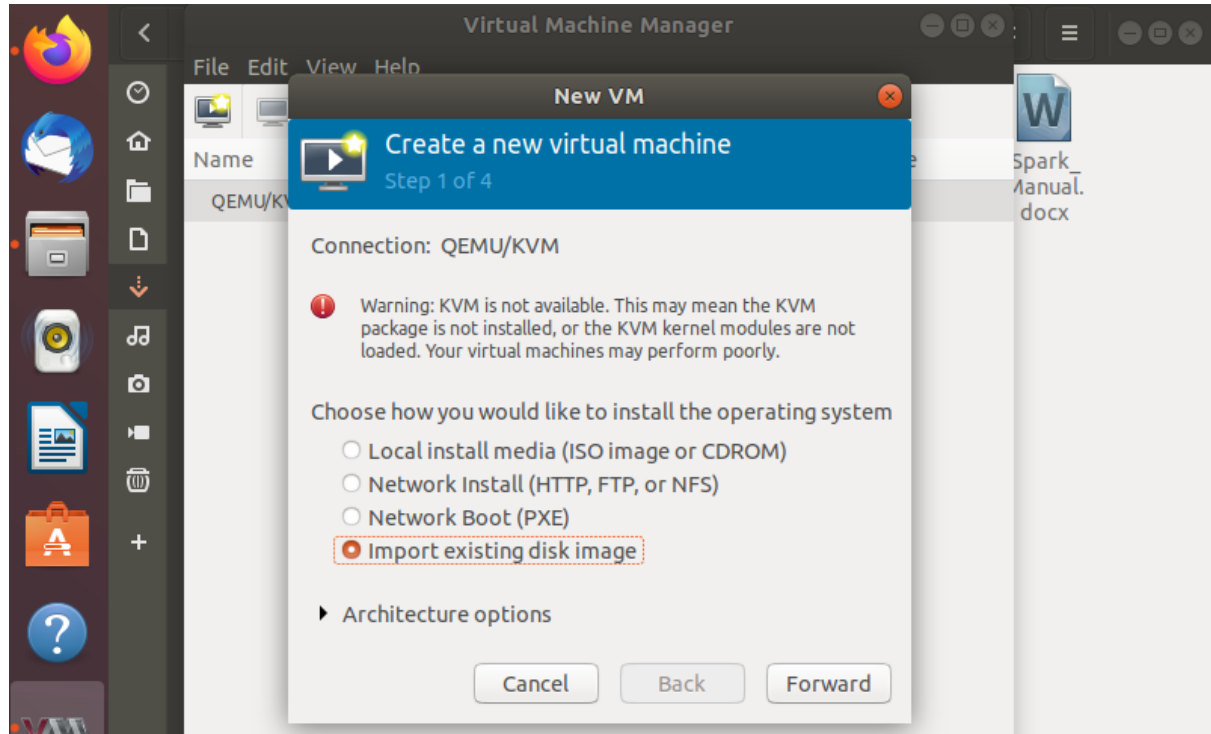
virsh #
```

Importing existing VM

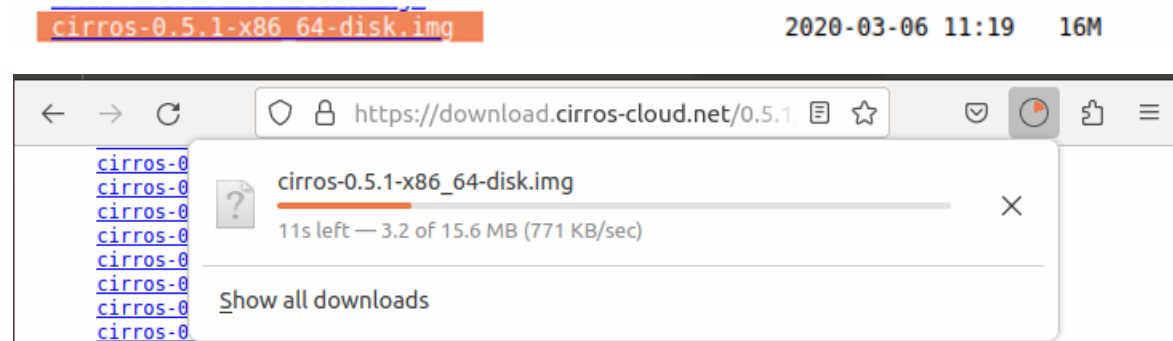
1. Go to virtual machine manager (virt-manager) in your linux system. Then, right click QEMU/KVM and select New to create new VM.



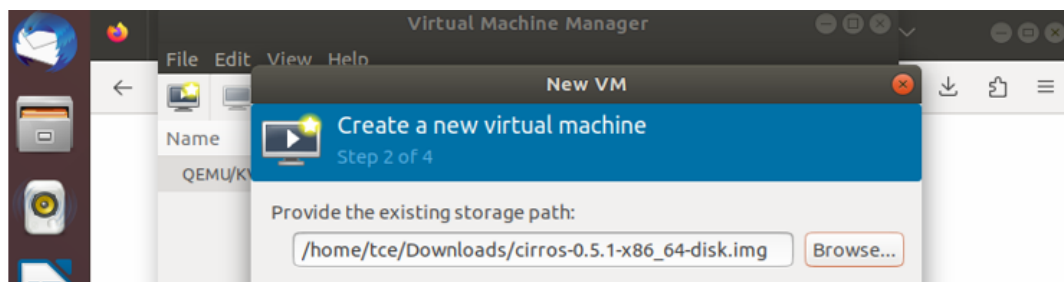
2. Select import existing disk image and click forward.



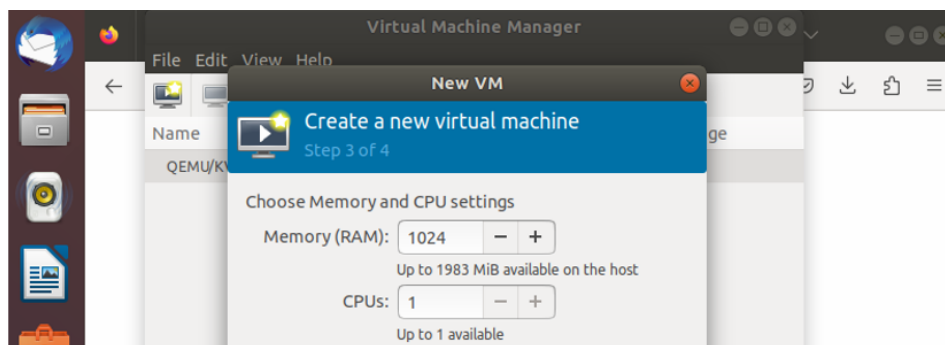
3. Download cirros disk image from the link: <https://download.cirros-cloud.net/0.5.1/>



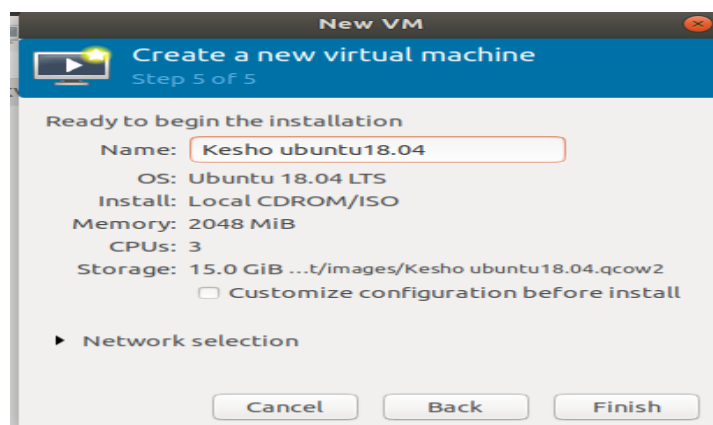
4. Give the path of downloaded cirros disk image and click forward.

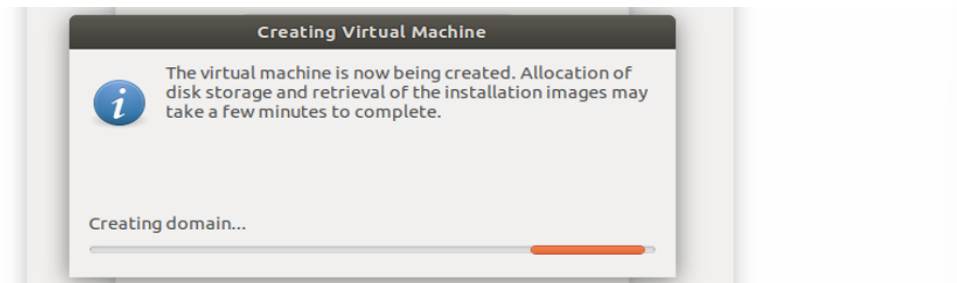


5. Choose memory and CPU settings and click forward.



6. Name your VM and click Finish.





7.The VM is booted successfully and the login was also successful.

```
File Virtual Machine View Send Key
[ 20.827594] PM: Magic number: 1:380:697
[ 20.850850] rtc_cmos 00:00: setting system clock to 2025-07-31T18:40:03 UTC (1753987203)
[ 20.876114] Unstable clock detected, switching default tracing clock to "global"
[ 20.876114] If you want to keep using the local clock, then add:
[ 20.876114] "trace_clock=local"
[ 20.876114] on the kernel command line
[ 21.025962] Freeing unused decrypted memory: 2040K
[ 21.167754] Freeing unused kernel image memory: 2660K
[ 21.173884] Write protecting the kernel read-only data: 22528k
[ 21.187794] Freeing unused kernel image memory: 2008K
[ 21.193999] Freeing unused kernel image memory: 1476K
[ 21.460992] x86/mm: Checked W+X mappings: passed, no W+X pages found.
[ 21.466577] Run /init as init process

further output written to /dev/ttyS0
[ 27.377055] random: fast init done
[ 31.112307] random: crng init done

login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
cirros login: cirros
Password:
$
```

Result:

Thus, the KVM was successfully installed on the Linux system, and virtual instances were created and configured using Virt-Manager, demonstrating the setup of a virtualized environment.

5b) KVM – Create Images from ISO, Image Resizing, and Image Conversion

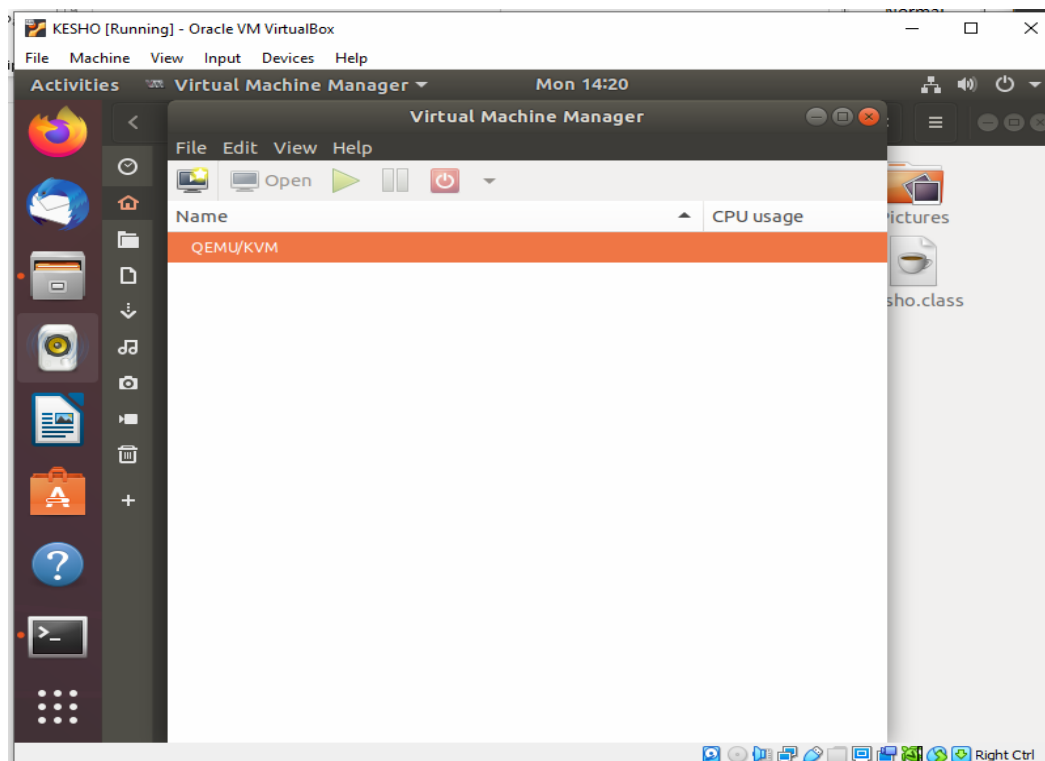
Aim:

To understand and demonstrate how to create virtual machine images using ISO files, resize virtual disk images, and convert disk images between various formats.

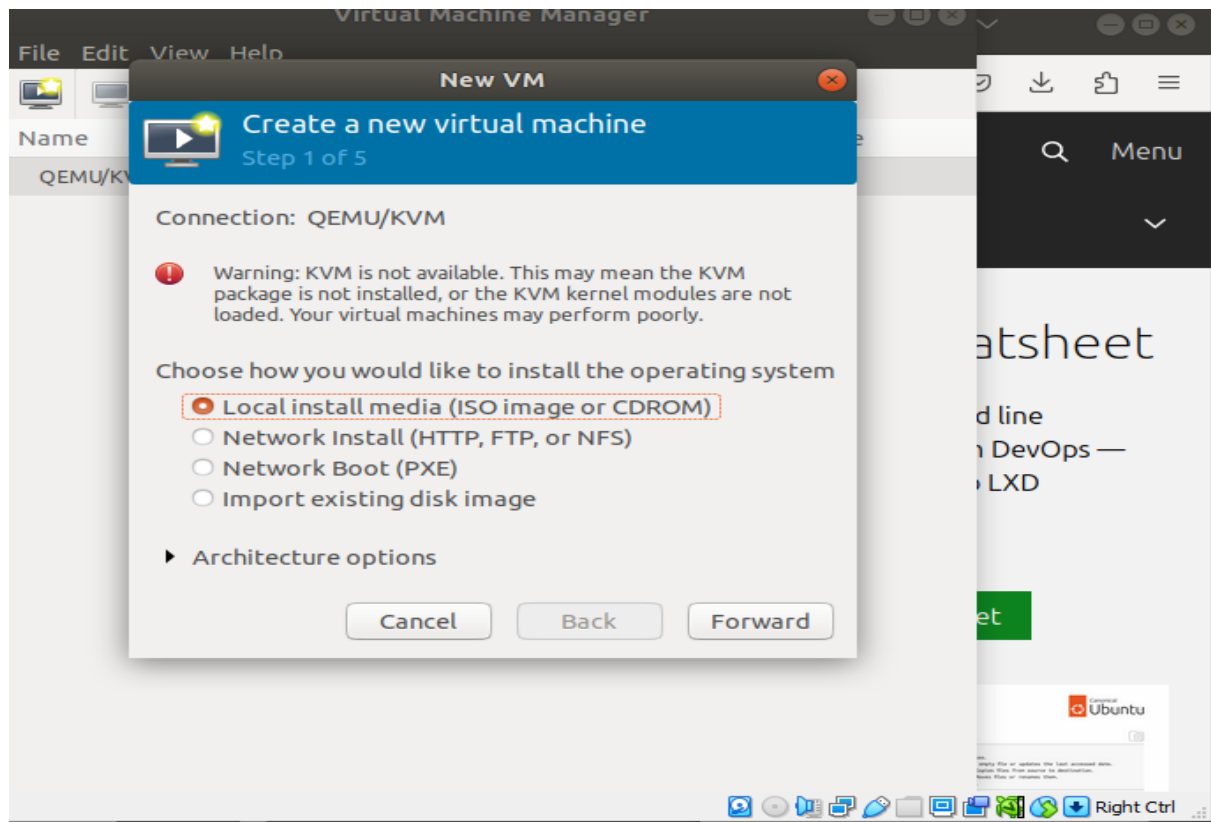
Procedure:

Creating VM image from ISO

1. Go to virtual machine manager (virt-manager) in your linux system. Then, right click QEMU/KVM and select New to create new VM.

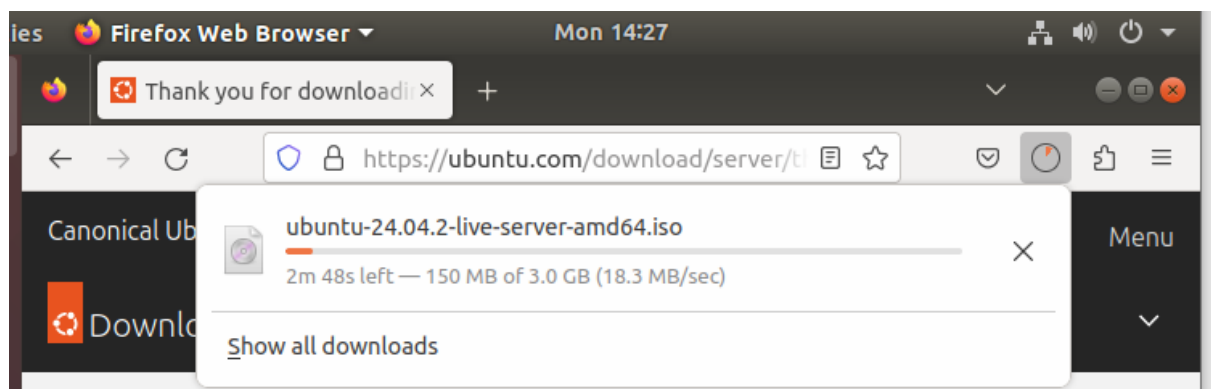


2. Select Local install media (ISO image or CDROM).

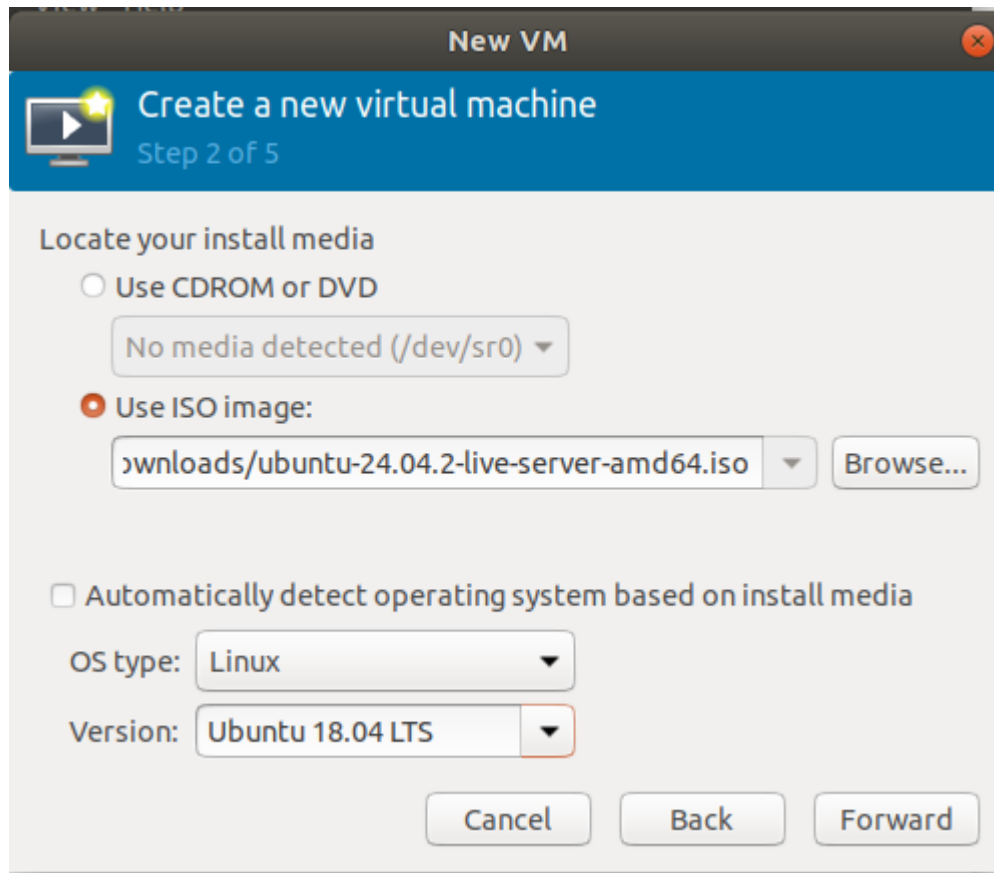


3. Download ubuntu server iso from the link:

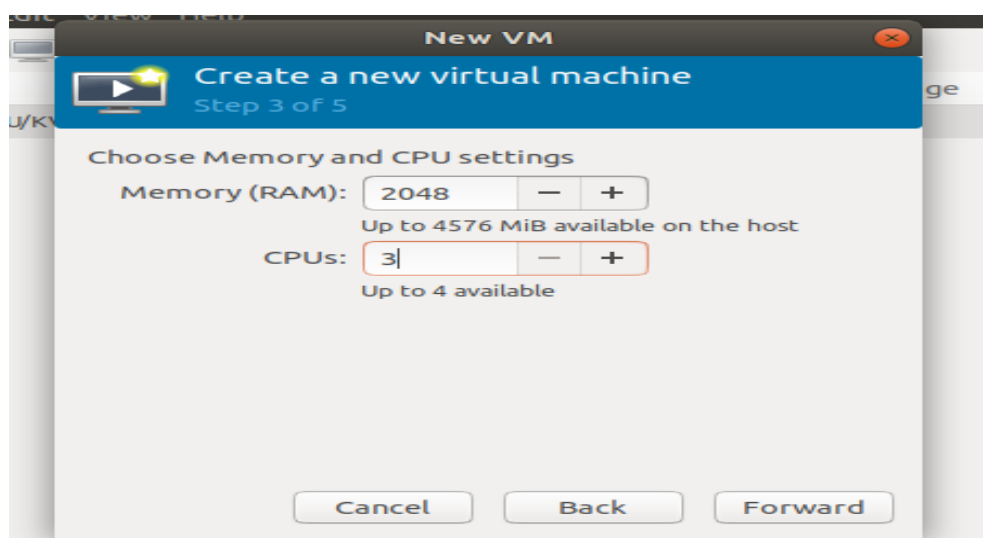
<https://releases.ubuntu.com/16.04/>



4. Select the downloaded ubuntu server iso path and click Forward.



5. Specify the memory and CPU required and click Forward.

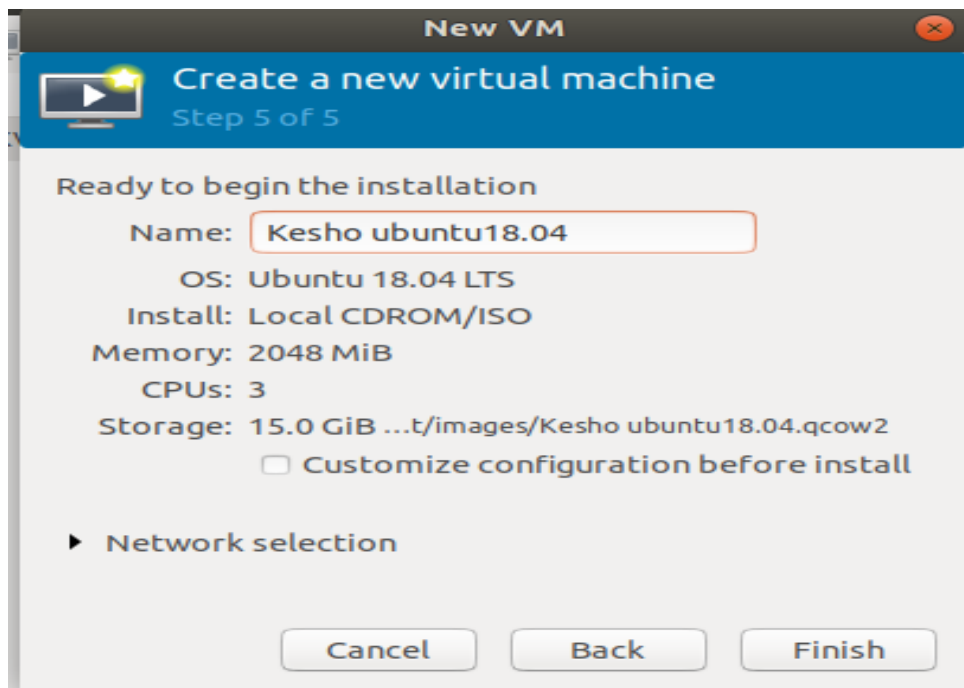


6.Run the command : *qemu-img create -f qcow2 ubuntu_vm.qcow2 10G*

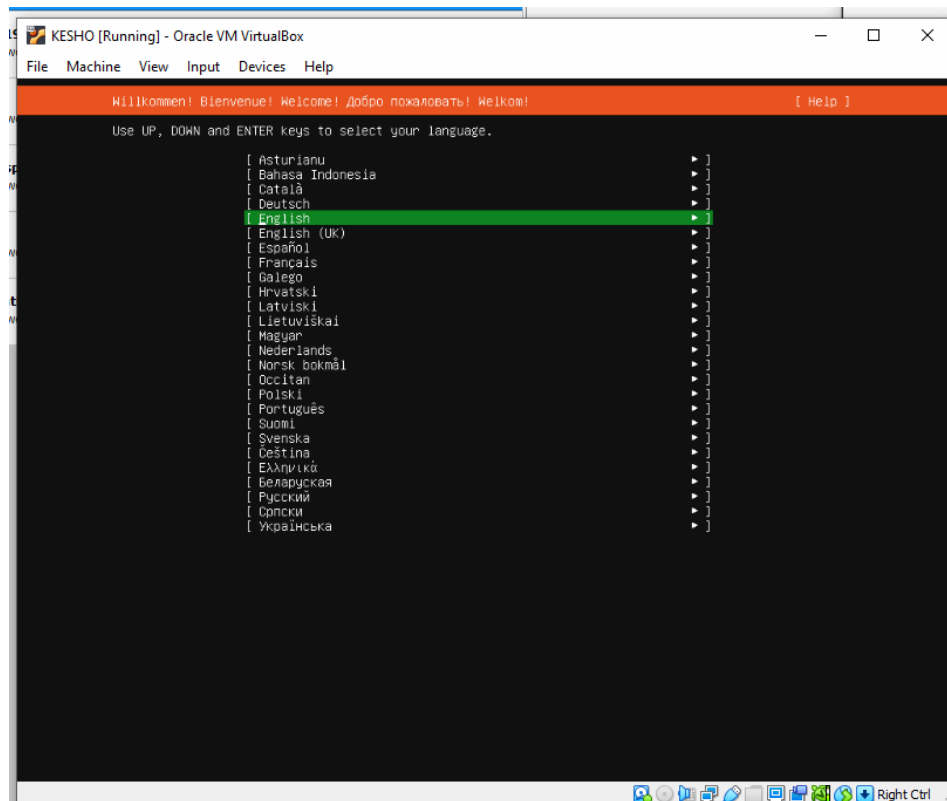
```
tce@tce-VirtualBox:~$ qemu-img create -f qcow2 ubuntu_vm.qcow2 10G
Formatting 'ubuntu_vm.qcow2', fmt=qcow2 size=10737418240 cluster_size=65536 lazy_refcounts=off refcount_bits=16
tce@tce-VirtualBox:~$
```

This command creates a virtual hard disk file (ubuntu_vm.qcow2) in the QCOW2 format, with a maximum size of 10 GB. This file will act as the virtual disk for a KVM virtual machine.

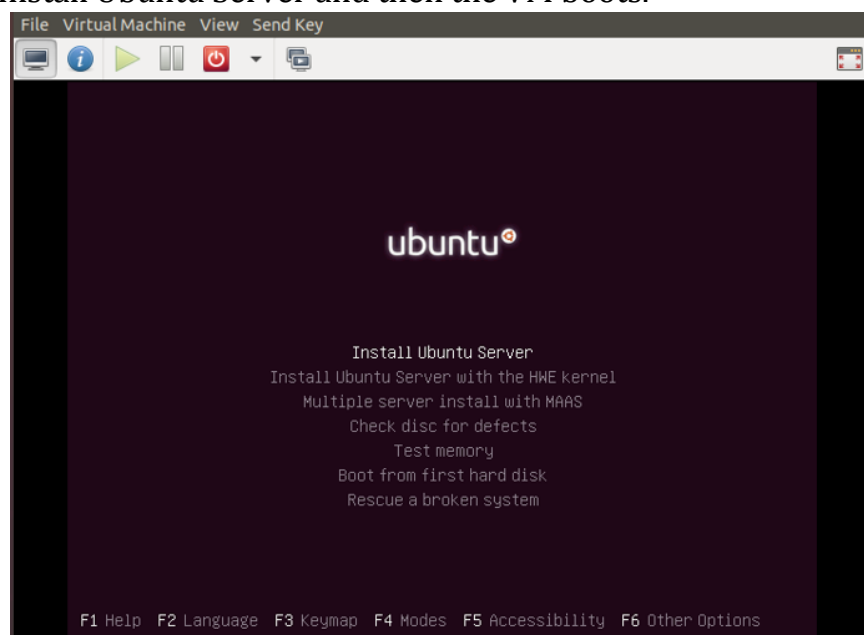
7.Name the VM and click Finish.



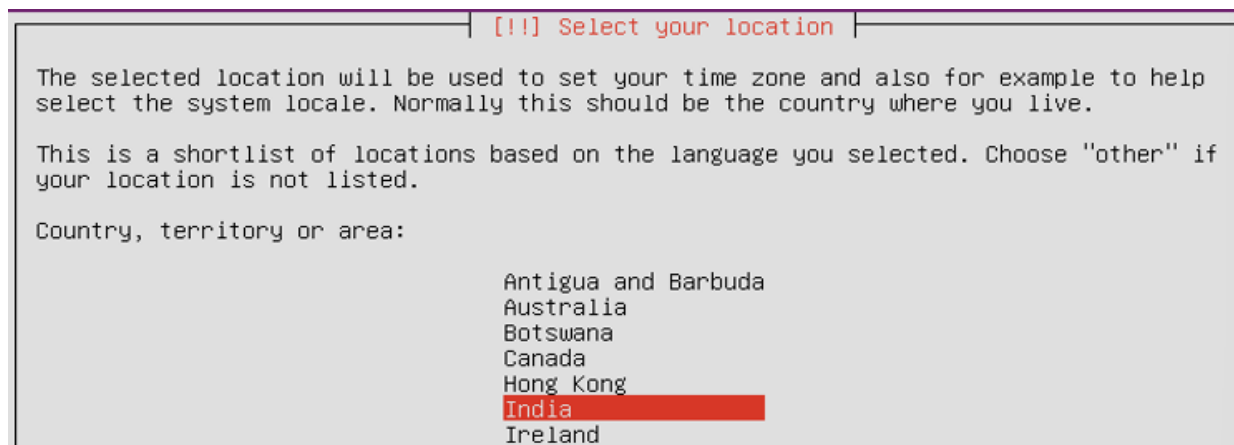
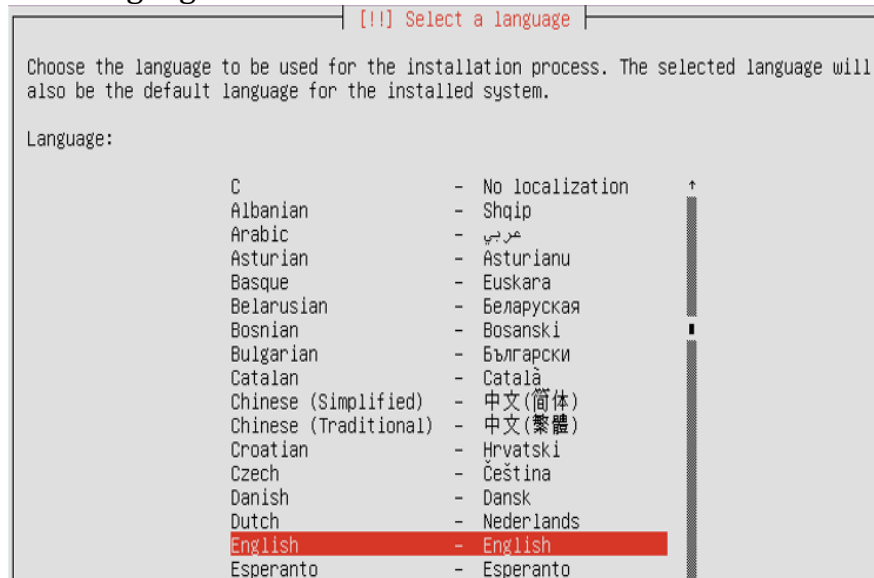
8. Select the language.



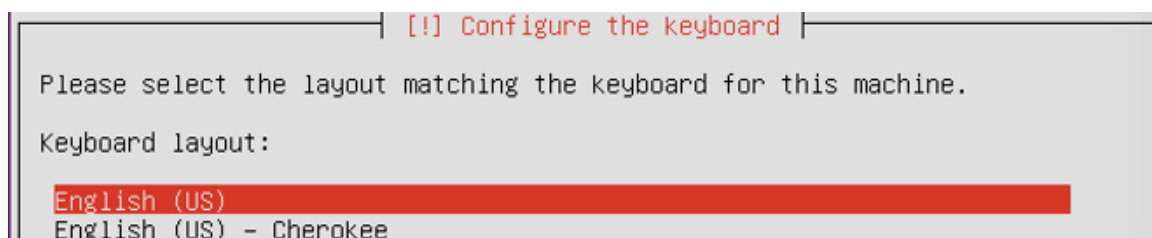
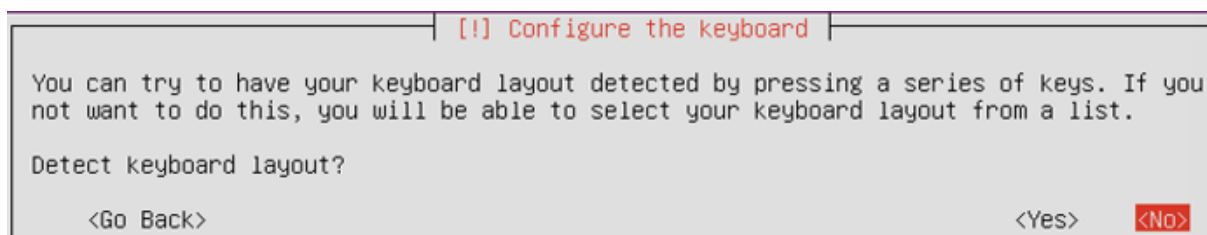
9. Select Install Ubuntu Server and then the VM boots.



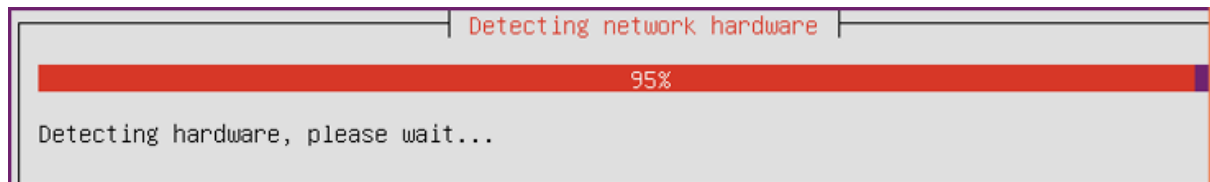
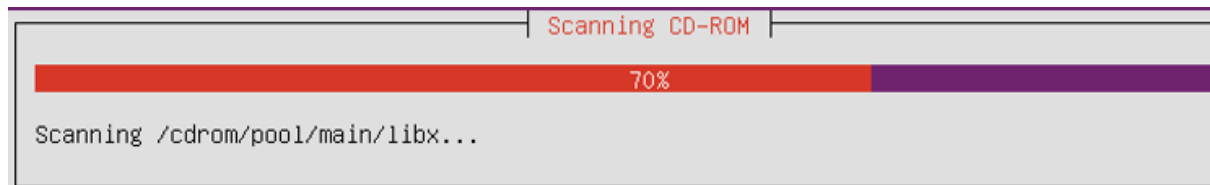
10. Select the language and location.



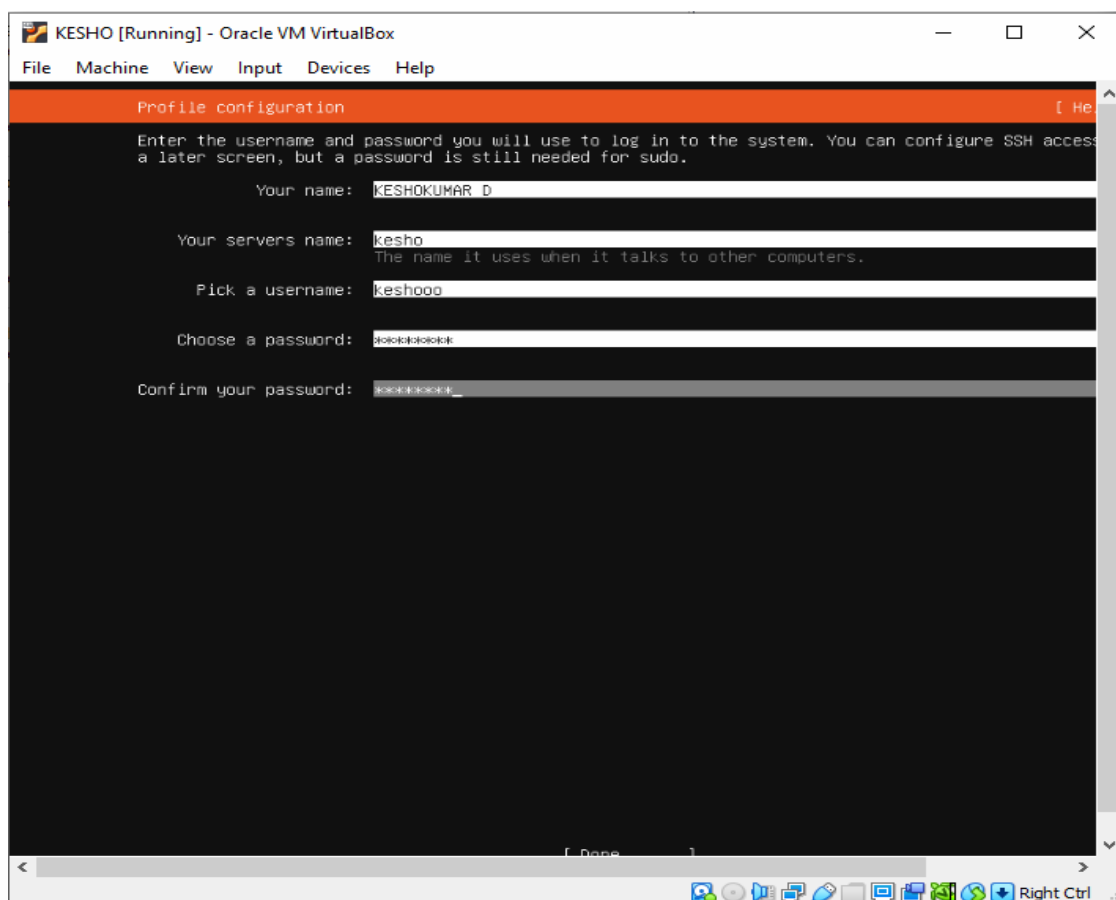
11. Set the Keyboard configuration.



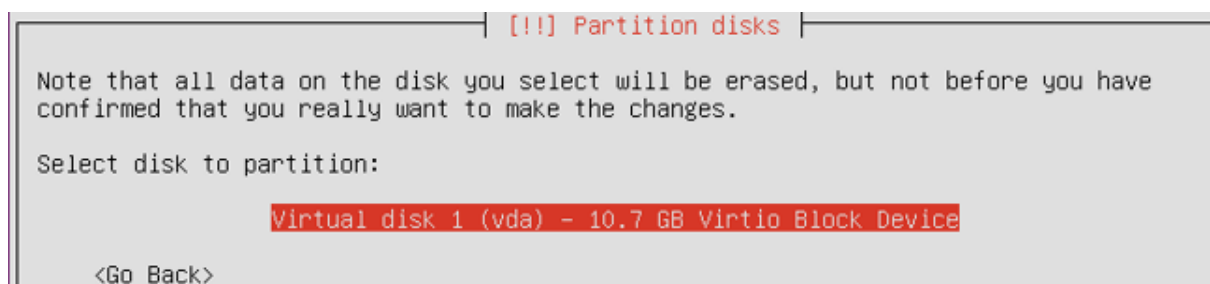
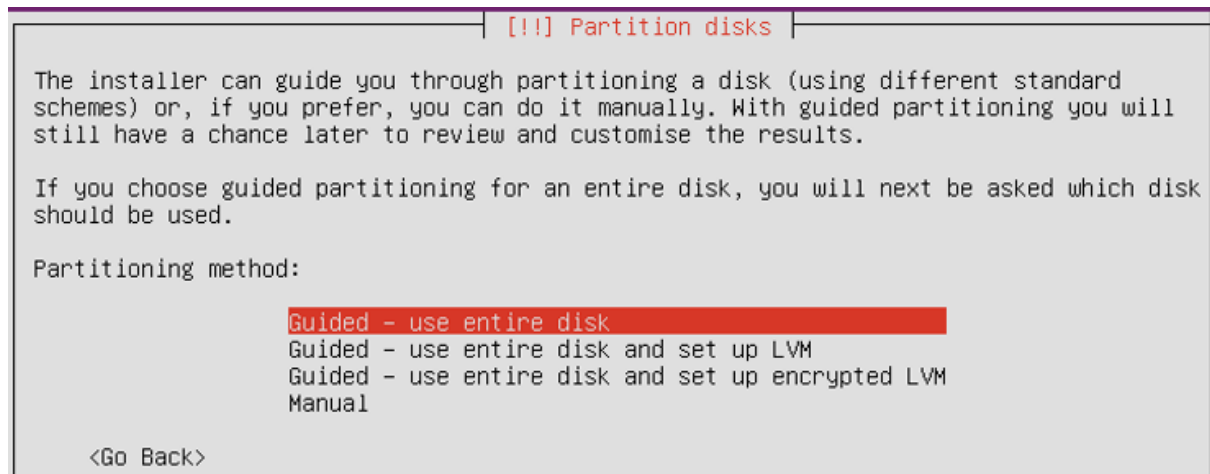
12. Then it starts detecting, loading and scanning of needed things.



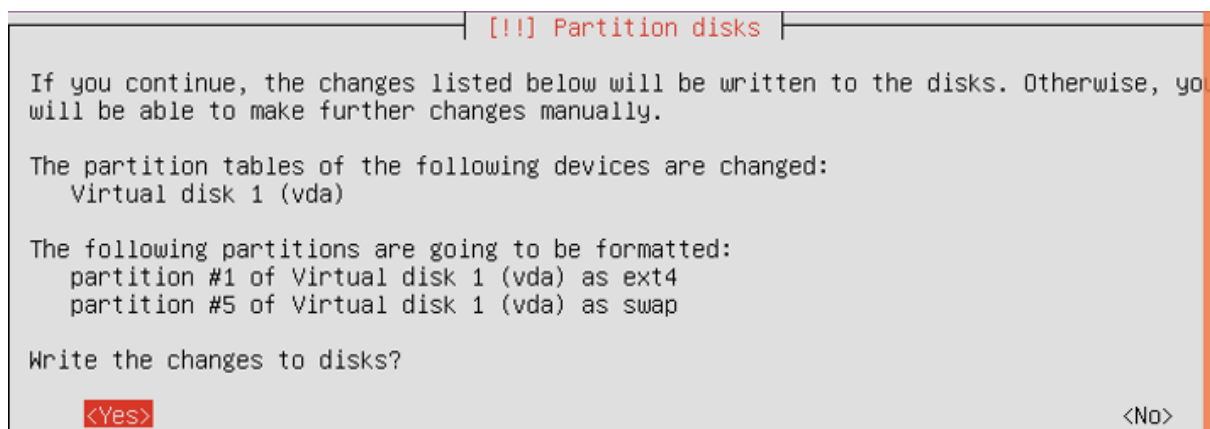
13. Then set hostname, username, password and clock settings.



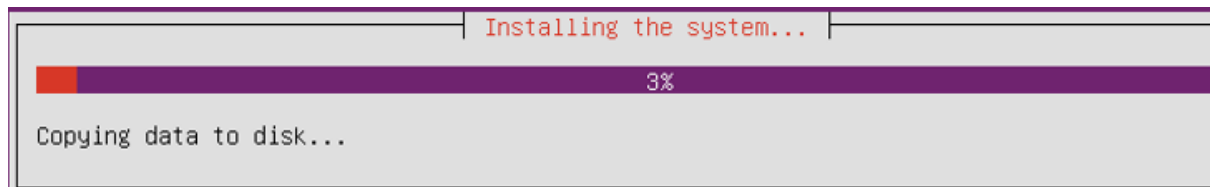
14. Set the disk partition.



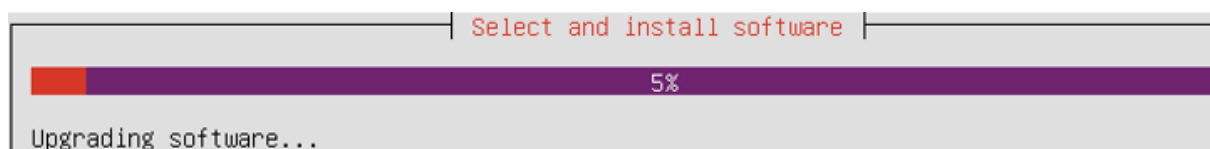
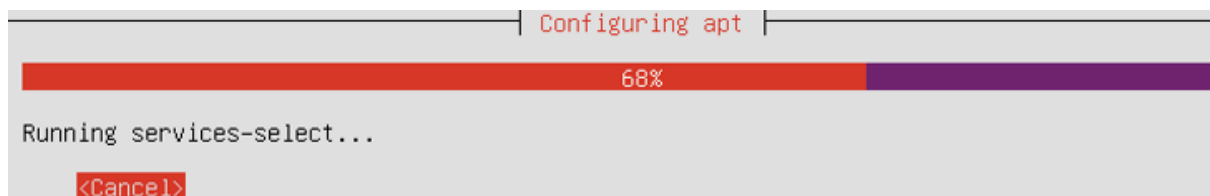
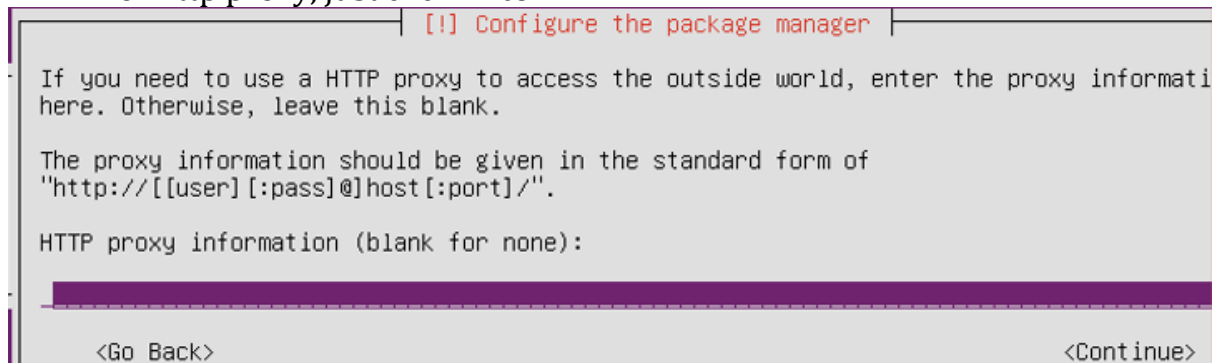
15. Select **Yes** to write the changes to disks.



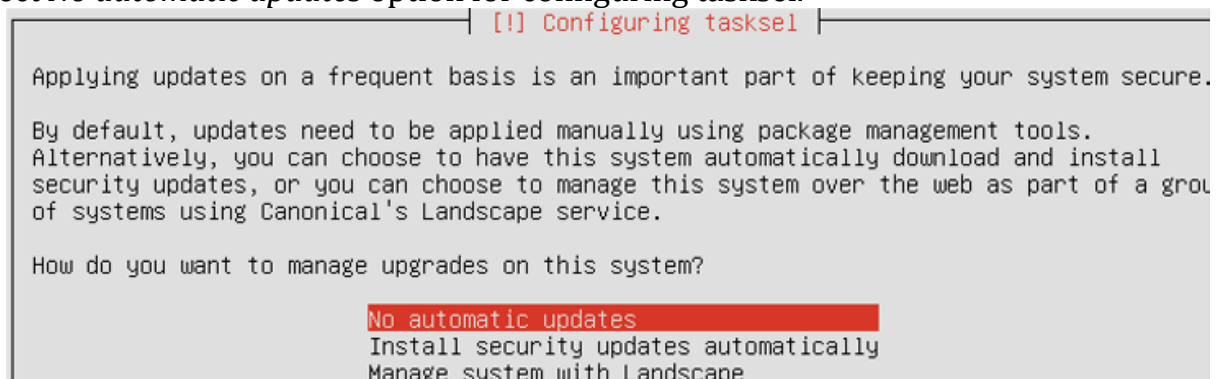
16.Then, the system installation starts.



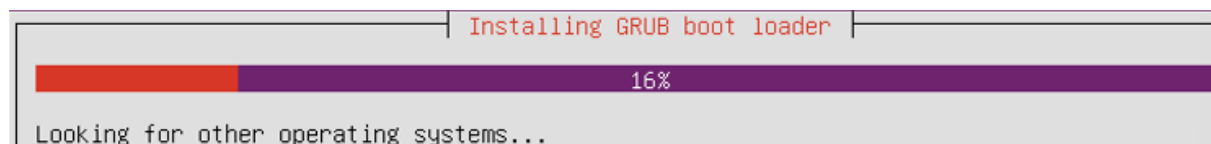
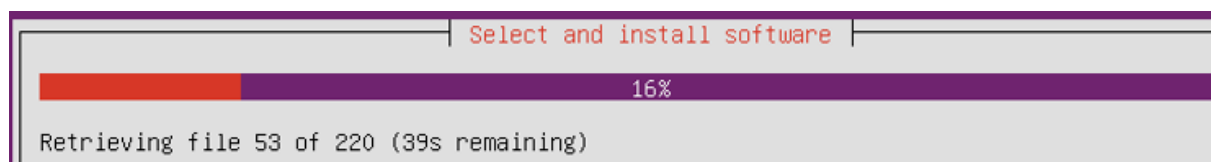
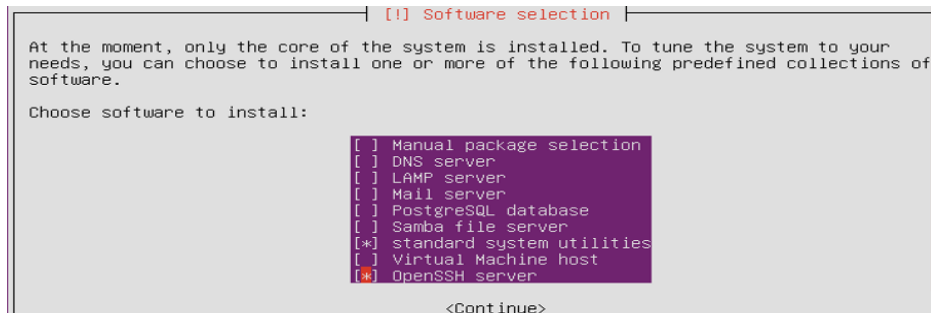
17.For http proxy, just click **Enter**.



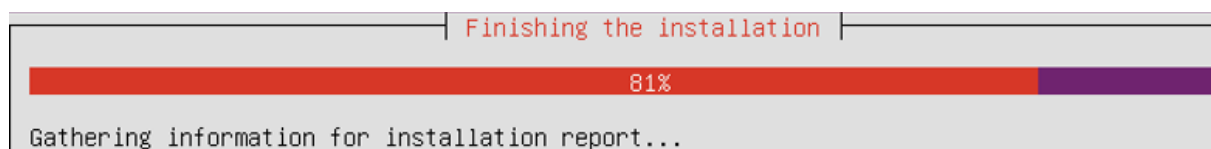
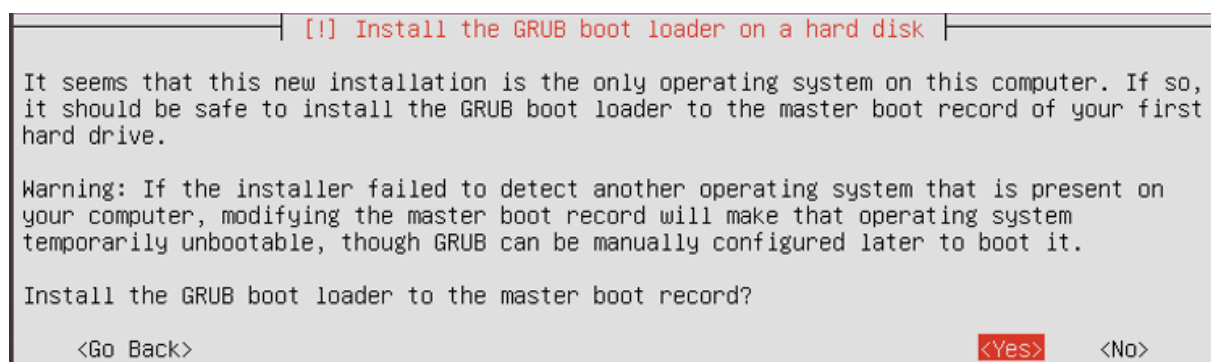
18.Select *No automatic updates* option for configuring tasksel.



19. In Software selection, select(use Spacebar) *standard system utilities* and *OpenSSH server*.



20. Give **Yes** to install GRUB boot loader.



21.Now, login to your VM.

```
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-142-generic i686)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage

194 packages can be updated.
138 updates are security updates.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

Image Resizing

1. Shut down the VM if it's running.
2. Resize the qcow2 image.

```
tce@tce-VirtualBox:~$ sudo qemu-img resize ubuntu_vm.qcow2 +10G
[sudo] password for tce:
Image resized.
tce@tce-VirtualBox:~$
```

```
tce@tce-VirtualBox:~$ qemu-img info ubuntu_vm.qcow2
image: ubuntu_vm.qcow2
file format: qcow2
virtual size: 20G (21474836480 bytes)
disk size: 2.0G
cluster_size: 65536
Format specific information:
  compat: 1.1
  lazy refcounts: false
  refcount bits: 16
  corrupt: false
tce@tce-VirtualBox:~$
```

3. Resize partitions inside the VM.

Once you've resized the image, boot into the VM and resize the partition/filesystem:

```
sudo growpart /dev/sda 1
```

```
sudo resize2fs /dev/sda1
```

Image Conversion

1. Conversion of qcow2 to vhd, vdi using qemu-img.

qcow2 to vdi

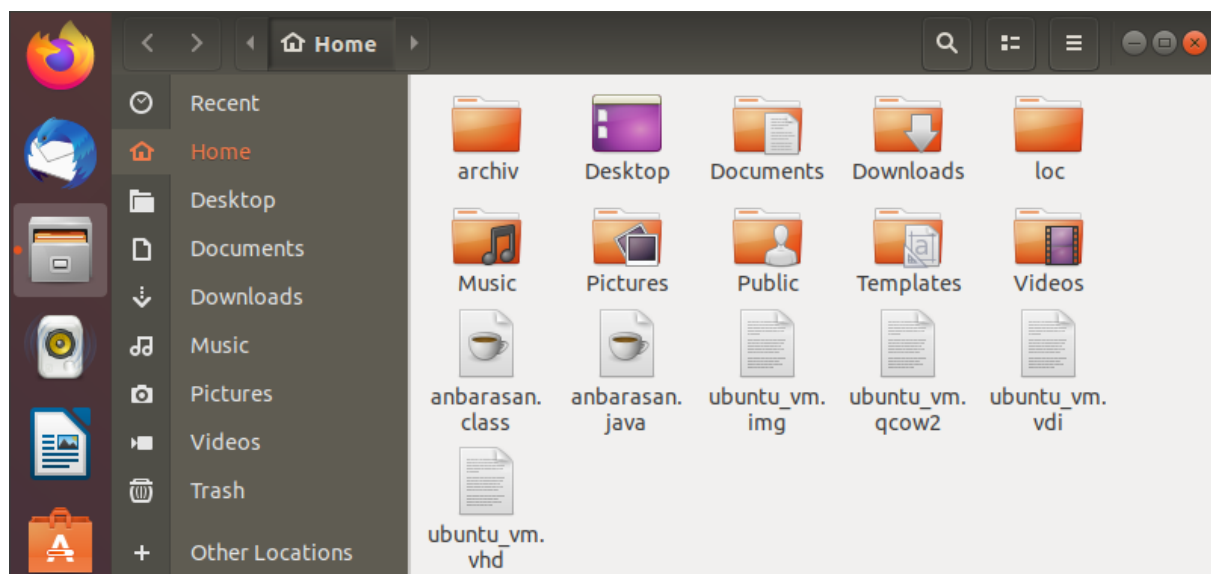
```
tce@tce-VirtualBox:~$ qemu-img convert -O vdi ubuntu_vm.qcow2 ubuntu_vm.vdi -p  
(100.00/100%)
```

qcow2 to vhd

```
tce@tce-VirtualBox:~$ qemu-img convert -O vpc ubuntu_vm.qcow2 ubuntu_vm.vhd -p  
(100.00/100%)  
tce@tce-VirtualBox:~$
```

qcow2 to img

```
tce@tce-VirtualBox:~$ qemu-img convert -O raw ubuntu_vm.qcow2 ubuntu_vm.img -p  
(100.00/100%)  
tce@tce-VirtualBox:~$
```



Result:

Thus, the creation, resizing, and format conversion of a KVM virtual disk image were successfully demonstrated.